GiTrip
~*Git* for Collaborative Trip Planning~

CS310 Computer Science Project

Tatsunori Ono

Supervisor: Professor Mike Joy

Second Assessor: Professor Charilaos Efthymiou

Department of Computer Science

University of Warwick

Academic Year 2025/26

Abstract

Trip planning becomes significantly more complex when multiple people collaborate, because itineraries evolve through parallel edits, last-minute changes, and disagreements about priorities. Existing tools typically optimise a single itinerary (e.g., mapping and routing applications) or support generic collaboration (e.g., shared documents), but they do not provide robust change-management semantics tailored to itineraries. This project presents *GiTrip*, a server-rendered web application that applies Git-style version control concepts to trip planning: a trip is modelled as a repository, alternative itineraries are branches, each saved version is a commit containing a complete snapshot, and divergent branches can be merged using a three-way merge with an explicit conflict-resolution user interface.

GiTrip integrates routing, scheduling constraints, and usability-focused interaction. An automatic scheduler produces feasible multi-day itineraries based on user inputs such as active hours, stay durations, ordering constraints, desired time windows, public transit selection, and opening hours. Users can then refine plans using two interactive surfaces: a timeline editor for adjusting arrival/departure times and a map editor for repositioning locations. To address the usability barrier of Git concepts for non-technical users, GiTrip includes an "Easy mode" that hides advanced controls while preserving version safety.

The completed system was evaluated through an in-person Evaluation Day demonstration and user acceptance testing. Objective task results show high completion rates for auto-scheduling feasibility and merge conflict resolution, with qualitative feedback highlighting the novelty and usefulness of version-controlled planning. The report details the requirements, system design, implementation, testing strategy, and evaluation outcomes, and concludes with limitations and future work (including optional machine learning extensions beyond the original project scope).

Keywords: trip planning; version control; branching and merging; scheduling constraints; routing; usability; progressive web app; conflict resolution

Acknowledgements

I would like to thank my supervisor, Professor Mike Joy, for consistent weekly guidance, constructive feedback on the Progress Report, and support in keeping the project aligned with the Project Specification. I also thank Professor Charilaos Efthymiou for assessment feedback that helped refine the evaluation approach and the presentation of technical material. Finally, I thank all participants who completed the preliminary survey and those who attended (or responded remotely to) the Evaluation Day survey and user acceptance tests; their time and feedback directly improved the usability and robustness of GiTrip.

Abbreviations

Term	Meaning
API	Application Programming Interface
CRDT	Conflict-free Replicated Data Type
CSP	Content Security Policy
DAG	Directed Acyclic Graph
DP	Dynamic Programming
EJS	Embedded JavaScript Templates
GDPR	General Data Protection Regulation
HTTP	Hypertext Transfer Protocol
iCal / ICS	iCalendar format / calendar file
JSON	JavaScript Object Notation
LCA	Lowest Common Ancestor (in a commit graph)
LLM	Large Language Model
NN	Nearest Neighbour (heuristic)
ORS	OpenRouteService
PWA	Progressive Web App
SQL	Structured Query Language
SUS	System Usability Scale
TAM	Technology Acceptance Model
TSP	Travelling Salesman Problem
UI/UX	User Interface / User Experience
WCAG	Web Content Accessibility Guidelines

Contents

Abstract **ii**

Acknowledgements **iii**

Abbreviations **iv**

1 Introduction **1**

1.1 Problem Context and Motivation 1

1.2 Project Overview 1

1.3 Objectives and Significance 2

1.4 Contributions 2

1.5 Report Structure 3

2 Background and Related Work **4**

2.1 Trip Planning and Routing Tools 4

2.2 Generic Collaboration Tools 4

2.3 Version Control Concepts and Applicability 5

2.4 Scheduling and Route Optimisation 5

2.5 Approximate String Matching for Place Search 5

2.6 Structured Data Merge and JSON Patch Formats 6

2.7 Usability and Accessibility Frameworks 6

2.8 Summary: Gap in Existing Approaches 6

3 Requirements and Project Specification **7**

3.1 Requirements Elicitation 7

3.2 MoSCoW Objectives 7

3.3 Success Criteria 8

3.4 Non-functional Requirements 8

3.5 Scope Boundary: Why AI Was Excluded from the MVP 9

4 Methodology **10**

4.1 Development Approach 10

4.2 Choice of Tools and Technologies 10

4.2.1 Node.js and Express 10

4.2.2 SQLite for Persistent Storage 11

4.2.3 Mapping and Geocoding 11

4.2.4 Routing Providers 11

4.2.5 Usability Frameworks 11

4.3 Testing Strategy 11

4.4 Evaluation Design 12

4.5 Ethics and Data Handling 12

5 System Design **13**

5.1 High-level Architecture 13

5.2 Data Model and Repository Semantics 14

5.2.1 Snapshot Storage 15

5.2.2 Plan Schema 16

5.2.3 Module Structure 16

5.3 Routing and Geocoding Design 17

5.3.1 Place Search 17

5.3.2 Routing Providers and Fallback 17

5.4 Auto-scheduler Design 18

5.4.1 Opening Hours Nudging 18

5.4.2 Compact vs Sparse Modes 18

5.5 Merge Design 18

5.5.1 Equality and Stability 20

5.5.2 Conflict Types 20

5.6 User Interface Design 20

5.6.1 Easy Mode vs Git Mode 20

5.7 Security, Privacy, and Accessibility Design 21

6 Implementation **22**

6.1 Backend Structure 22

6.1.1 Database Layer 22

6.1.2 Error Handling 22

6.2 Planner Implementation . . . 23
6.3 Merge Implementation . . . 25
6.4 Routing Implementation . . . 25
6.5 Quick Explore Mode . . . 25
6.6 Algorithm Visualisation . . . 26
6.7 Frontend Implementation . . . 26
6.7.1 Timeline and Map Editing . . . 26
6.8 Exports and PWA Features . . . 27
6.8.1 PDF Export . . . 27
6.8.2 iCalendar Export . . . 27
6.8.3 PWA and Offline . . . 27
6.9 Supporting Features . . . 27
6.9.1 Weather Integration . . . 27
6.9.2 Travel Alerts . . . 27
6.9.3 Public Gallery with Pagination . . . 27
6.10 Deployment and Operations . . . 28

7 Testing **29**
7.1 Unit Testing . . . 29
7.1.1 Opening Hours Parsing and Validation . . . 29
7.1.2 Merge Conflict Detection . . . 30
7.1.3 Planner Time Utilities and Scheduling Logic . . . 30
7.1.4 Routing Fallback Logic . . . 30
7.1.5 External API Connectivity . . . 31
7.2 Functional Testing . . . 31
7.3 Non-functional Testing and Checks . . . 32
7.3.1 Security Checks . . . 32
7.3.2 Accessibility Checks . . . 32
7.3.3 Resilience to External API Failures . . . 33
7.4 Testing Summary . . . 33

8 Evaluation **34**
8.1 Evaluation Day Setup . . . 34
8.2 Participants . . . 34
8.3 User Acceptance Testing Tasks . . . 34
8.3.1 Task T1: Auto-schedule Feasibility . . . 35
8.3.2 Task T2: Merge Conflict Resolution . . . 35

8.3.3 Task T3: Quick Edit + Save Without Git Explanation 36
8.4 Survey Results 36
8.4.1 Qualitative Feedback 36
8.5 Discussion Against Success Criteria 37
8.6 Threats to Validity 37

9 Conclusions and Future Work 38
9.1 Conclusions 38
9.2 Critical Reflection and Lessons Learned 38
9.3 Limitations 39
9.4 Future Work 39

Author's Assessment of the Project 41

A Evaluation Day Survey Instrument and Results 42
A.1 Survey Questions (Summary) 42
A.2 Full Survey Export PDF 42

B User Acceptance Test Results (Full Tables) 43

C Additional Diagrams and Screenshots 44

D Project Schedule 45

List of Figures

3.1 Preliminary survey evidence informing MoSCoW objectives (placeholder). 7

5.1 GiTrip high-level architecture. The browser communicates with the Express server over HTTP/JSON. *UI Routes* render pages while *API Routes* delegate to the *Planner*, *Merge*, *Routing*, and *Geo Search* modules. *Quick Route* delegates to *Routing*, which in turn calls ORS for non-transit modes and Google Directions for transit legs. The *Planner* consults the *Opening Hours Parser*, a local utility that validates time intervals against OSM opening-hours strings supplied by Nominatim via *Geo Search*. Authentication middleware enforces session management, rate-limiting, and Content Security Policy headers across all routes. Server route handlers fetch weather forecasts from Open-Meteo and travel alerts from GNews. All persistent state is stored in a SQLite database. 14

5.2 Entity relationship diagram for GiTrip's data model using Crow's foot notation. Primary keys are <u>underlined</u>. Solid-bordered entities are persisted as SQLite tables; dashed-bordered entities represent JSON structures embedded within the `snapshot` column of each Commit. Crow's foot endpoints: double bar = exactly one, circle–bar = zero or one, fork–bar = one or many, fork–circle = zero or many. 15

5.3 UML class diagram for GiTrip's server-side modules. Each module is shown with public (`+`) and private (`-`) operations. Dashed open arrows denote ⟪use⟫ dependencies; the filled diamond denotes composition (Server owns TTLCache instances). `TTLCache` is a formal JavaScript class; all other boxes are CommonJS modules. 17

5.4 Three-way merge decision flowchart. The engine iterates all file keys, recording conflicts or taking the changed side, then checks plan divergence. If both sides changed the plan identically, either is taken. If both changed the plan structure and their stop-key Jaccard similarity is below 0.4, a whole-plan conflict is raised. Otherwise, per-stop merging chains through deletion, time, and field checks. The result is stabilised by deterministic ordering before output. 19

6.1 Auto-scheduler flowchart. The algorithm builds an ORS travel-time matrix (with Haversine fallback), orders stops (strict constraints or Nearest Neighbour), then partitions them into preferred-day buckets and an unassigned pool. Phase 1 schedules preferred-day stops on their assigned day, sending unfitting stops directly to overflow. Phase 2 schedules unassigned stops greedily, spilling to subsequent days (retrying the same stop) or overflow when no days remain. The cursor is initialised with a focus-dependent shift (morning, midday, or afternoon). 24

C.1 Merge conflict resolution UI. 44

C.2 Timeline editor with draggable stop blocks. 44

C.3 Map editor with numbered markers and route geometry. 44

C.4 Easy mode vs Git mode comparison. 44

C.5 Algorithm visualisation page. 44

C.6 Quick explore mode with optimised route. 44

D.1 Project Gantt chart showing the timeline for each development phase. Phases overlap where integration dependencies allowed parallel work. Key milestones are marked above the relevant bars. 45

List of Tables

7.1 Summary of unit test coverage (64 test cases). 29
7.2 Functional test cases for core workflows (summary). 32

8.1 UAT Task T1 — Auto-schedule feasibility. 35
8.2 UAT Task T2 — Merge conflict resolution. 35
8.3 UAT Task T3 — Quick edit and save without Git explanation. 36

D.1 Schedule detail: deliverables and status per phase. 45

Chapter 1

Introduction

This report presents the design, implementation, testing, and evaluation of *GiTrip*, a web application for collaborative trip planning with Git-like version control semantics. The core premise is that itineraries change in ways similar to software artefacts: multiple contributors propose alternatives, edits happen in parallel, and the history of decisions matters. In conventional trip-planning workflows, these properties are handled informally through screenshots, chat messages, and manual copy-pasting across documents, which makes the process fragile and difficult to audit.

1.1 Problem Context and Motivation

Trip planning is inherently multi-constraint: a feasible itinerary must account for travel time between destinations, opening hours, stay durations, and daily activity limits. For group trips, coordination adds an additional dimension: multiple people make edits simultaneously, propose alternatives, or revise plans after new information emerges. The absence of a robust change-management model leads to common failure modes, such as overwritten edits, confusion about what changed and why, and difficulty exploring alternatives without destabilising the "main" plan.

Mapping and routing tools such as Google Maps support route optimisation but largely focus on a single itinerary at a time and do not provide structured collaborative versioning for itinerary artefacts [**?**]. Generic collaboration tools such as Google Docs can support co-authoring, but they provide limited structure for route and schedule constraints, and they do not provide domain-specific merge semantics for itineraries [**?**]. GiTrip addresses this gap by combining: (i) itinerary-aware scheduling and routing; (ii) Git-inspired branching and merging; and (iii) an interaction model that remains accessible to users who do not know Git.

1.2 Project Overview

GiTrip is a server-rendered web application (Node.js + Express) that models a trip as a repository and stores each saved version as a commit containing a full snapshot of the trip plan. Users can create branches to explore alternate itineraries, compare versions, and merge changes with a three-way merge. When conflicts occur, GiTrip provides a dedicated merge user interface that

allows users to choose resolutions rather than silently overwriting information.

The system additionally includes:

- An automatic scheduler that generates a feasible multi-day itinerary using routing durations and user constraints.
- Interactive plan editing via a timeline editor (arrival/departure times) and a map editor (coordinates).
- "Easy mode" UX that hides Git-specific vocabulary and advanced controls for non-technical users.
- A public gallery and star mechanism for browsing and lightweight social interaction.
- Export features (PDF via browser print, iCalendar export).
- Progressive Web App (PWA) capabilities and offline support.

1.3 Objectives and Significance

The project objectives were initially formalised using a MoSCoW prioritisation in the Project Specification (Must/Should/Could/Won't). The Must-have objectives included repository history, an optimised automatic scheduler, routing integration, interactive visualisation, and merging with conflict resolution. These objectives were chosen because they jointly define GiTrip's distinctive contribution: no single competitor product simultaneously provides (a) itinerary scheduling with constraints, (b) collaborative change history, and (c) Git-style branching and merging semantics in the trip-planning domain [?, ?, ?, ?, ?].

A key boundary condition is that the original scope intentionally excluded AI-powered itinerary generation from natural language. This decision was stated as a Won't-have requirement to prioritise deterministic, inspectable scheduling behaviour, and to avoid expanding scope into content recommendation and personal preference modelling. The second assessor's Progress Report feedback suggested investigating machine learning to strengthen the project; this report therefore discusses AI as future work while justifying why it was not appropriate for the Minimum Viable Product (MVP) scope.

1.4 Contributions

The technical contributions of this project are:

- A repository and snapshot-based storage model for itinerary versions, enabling fast branch switching and reproducible restores.
- A three-way merge engine for itinerary snapshots, including conflict detection for time clashes, deletions, and selected non-time fields, plus a corresponding conflict-resolution UI.
- A constraint-aware auto-scheduler that integrates travel-time estimation, opening-hours nudging, desired windows, and spillover across days.

- A dual-path user interface (Easy mode vs Git mode) supporting both novice and power-user workflows.
- A deployed web system with practical security measures (sessions, CSP, rate limiting) and supporting features (weather, travel alerts, checklists).

1.5 Report Structure

Chapter 2 reviews related tools and foundational concepts. Chapter 3 describes requirements and the rationale for the MoSCoW objectives. Chapter 4 outlines the development methodology and evaluation strategy. Chapters 5 and 6 present the system design and implementation. Chapter 7 summarises the testing strategy and evidence. Chapter 8 reports Evaluation Day results and survey findings. Chapter 9 concludes with limitations and future work, followed by appendices containing supporting evidence.

Chapter 2

Background and Related Work

This chapter positions GiTrip within existing trip-planning and collaboration tooling, and introduces foundational techniques used in the system: version control semantics, scheduling heuristics, string matching, accessibility heuristics, and relevant web standards.

2.1 Trip Planning and Routing Tools

Most users plan trips using mapping and routing tools such as Google Maps and Apple Maps [**?**,**?**]. These applications provide high-quality route computation and support multiple transport modes, but their data model is fundamentally *single-plan centric*. Users can create lists and saved routes, but they cannot easily branch into alternative itineraries, compare versions, and merge parallel edits with structured conflict handling.

Public-transport-focused tools such as Citymapper offer strong transit routing and navigation [**?**], but similarly treat the itinerary as an output rather than a version-controlled artefact. Trip planning products such as Wanderlog, Furkot, Roadtrippers, and TripIt provide itinerary management features and collaboration to varying degrees [**?**,**?**,**?**,**?**], yet they generally emphasise itinerary presentation and booking integration rather than explicit version control semantics. Google Travel aggregates trip components and suggestions but does not provide Git-style change management [**?**].

GiTrip's differentiation is therefore not *better routing* than these tools; instead, it provides structured change management and alternative exploration, paired with sufficient routing and scheduling integration to keep plans feasible.

2.2 Generic Collaboration Tools

When collaboration is required, users commonly use documents and spreadsheets to manage itineraries: Google Docs/Sheets, Microsoft Word/Excel, and increasingly general workspaces like Notion [**?**,**?**,**?**,**?**,**?**]. These platforms have excellent collaboration features but treat an itinerary as unstructured text or loosely structured tables. As a result, they cannot automatically recompute travel times after edits, offer interactive map or timeline visualisation, detect domain-specific conflicts such as overlapping times, or provide branching semantics for exploring alternatives.

GiTrip's objective is to bring the strengths of version control (branching/merging/history) into a domain-specific planner where plans are first-class structured data.

2.3 Version Control Concepts and Applicability

Git is the dominant distributed version control system in software development [**?**]. Its key semantics include commits (immutable versions), branches (named pointers to commits), and merges (reconciling divergent commit histories). Git's internal model stores snapshots rather than deltas: each commit represents a complete view of the repository state [**?**]. This is particularly relevant for GiTrip because a trip plan can be represented naturally as a JSON document. Snapshot storage enables instant branch switching (render the head snapshot), version restoring (create a new commit from an older snapshot), and three-way merges (compare a base snapshot against two branch snapshots).

However, applying Git semantics to structured itinerary data introduces challenges: traditional line-based diff/merge strategies (e.g., `diff3`) are designed for text and can produce poor results for structured JSON. The fundamental three-way merge idea remains useful: derive changes in two branches relative to a common ancestor and combine them, surfacing conflicts when both sides change the same part incompatibly. Formal work on `diff3` analyses its behaviour as a widely used three-way merge algorithm [**?**].

GiTrip therefore adapts the three-way merge concept to structured itinerary objects rather than line-based text.

2.4 Scheduling and Route Optimisation

A key sub-problem in trip planning is ordering stops efficiently. This relates to the Travelling Salesman Problem (TSP), which is computationally hard in general and commonly addressed with heuristics [**?**, **?**]. GiTrip uses heuristics that provide good-enough results for interactive planning rather than exact optimal solutions, which aligns with practical timetabling and scheduling approaches discussed in the automated timetabling literature [**?**].

For example, the Nearest Neighbour (NN) heuristic greedily selects the closest next stop and is a common baseline; it is known to be simple and fast while not always optimal [**?**]. GiTrip also includes an educational algorithm visualisation that demonstrates NN, NN + 2-opt improvement, and Held-Karp dynamic programming for small instances, helping explain why the production scheduler uses heuristics.

2.5 Approximate String Matching for Place Search

Place search relies on external geocoding and search services (e.g., Nominatim). Returned results can be noisy or differently formatted across user inputs. GiTrip improves usability by re-ranking candidates using approximate string matching, normalising input strings and applying edit distance techniques [**?**]. This improves the "easy add" flow where users paste free-form place names and expect robust matching.

2.6 Structured Data Merge and JSON Patch Formats

JSON-based systems frequently represent updates using patch formats. RFC 7396 (JSON Merge Patch) defines a merge-based approach for describing modifications to JSON objects [**?**]. However, general-purpose patch formats do not provide domain-specific semantics for itineraries: for example, they do not encode constraints such as time overlaps or opening-hours validity. GiTrip therefore uses a domain-aware merge strategy:

- file-like maps (`snapshot.files`) are merged per-key using deep equality,
- trip plans (`snapshot.plan`) are merged per-day and per-stop with explicit conflict detection for semantic fields.

2.7 Usability and Accessibility Frameworks

GiTrip must balance power-user semantics (branches, merges) with novice usability. Nielsen's usability engineering framework provides well-established heuristics for evaluating interfaces and diagnosing common usability problems [**?**]. Nielsen's heuristics were preferred over heavier frameworks because they are practical for small teams, support rapid iteration, and directly map to UI design decisions such as visibility of system state, consistency, error prevention, and recognition over recall.

Accessibility was guided by WCAG 2.2 principles [**?**], particularly around keyboard navigation, contrast, and clear labelling. GiTrip's use of interactive map and timeline components introduces accessibility challenges; this report therefore discusses mitigations and remaining limitations.

For evaluation, GiTrip used a survey instrument that included the System Usability Scale (SUS) and Likert-style questions aligned with constructs from the Technology Acceptance Model (TAM), notably perceived usefulness and perceived ease of use [**?**]. TAM was not used as a strict statistical model due to limited sample size, but its constructs informed question design and interpretation.

2.8 Summary: Gap in Existing Approaches

In summary, existing trip planning tools provide routing and itinerary management but lack robust version control semantics. Existing collaboration tools provide sharing but lack domain structure. Version control systems provide branching and merging but do not operate on itinerary semantics. GiTrip's novelty is that it combines these approaches into a coherent system: a version-controlled, constraint-aware, user-editable itinerary planner.

Chapter 3

Requirements and Project Specification

This chapter describes the requirements that guided GiTrip's development, how they were derived, and how they map to success criteria and evaluation. It also documents scope boundaries, including the explicit decision to exclude AI-powered itinerary generation from the MVP.

3.1 Requirements Elicitation

Requirements were formed using two complementary inputs:

- **Background research** into competitor tools (maps, planners, collaboration platforms) and foundational techniques (version control, scheduling heuristics).
- **A preliminary survey** (n=30) conducted via Microsoft Forms (Nov 24–27, 2025) to understand current trip-planning workflows, perceived pain points, and interest in proposed features.

The preliminary survey provided evidence that trip planning is often stressful/time-consuming and that users rely on a fragmented toolkit (maps, documents, chats). Importantly, feedback indicated a desire for both automation (route/schedule generation) and change management (saving versions, exploring alternatives). The survey also highlighted that Git vocabulary can be intimidating for non-technical users, motivating the Easy mode design and the emphasis on clear visualisation.

Placeholder: summary chart(s) from the preliminary survey (Appendix A).

Figure 3.1: Preliminary survey evidence informing MoSCoW objectives (placeholder).

3.2 MoSCoW Objectives

The Project Specification defined MoSCoW objectives. The Must-have objectives (M1–M5) formed the MVP definition:

- **M1 Trip repositories with history:** commits, branches, and snapshot history; visualisation of version history.
- **M2 Optimised automatic scheduler:** produce a feasible multi-day plan from places, durations, ordering constraints, desired windows, active hours, breaks, target day ranges, and preferences.
- **M3 Routing integration:** retrieve travel-time data (e.g., OpenRouteService and Google Directions) and feed into scheduling.
- **M4 Interactive visualisation:** timeline and map with editing and committing.
- **M5 Merge and conflicts:** structured merge for plan JSON with a conflict resolver screen.

Should-have objectives included opening-hours validation and preference handling. The completed system implements these via opening-hours parsing and nudging, and compact vs sparse scheduling modes.

Won't-have objectives included real-time collaborative editing (CRDT/OT), booking integration, budgets, social chat/reviews, native apps, and AI-powered itinerary generation. These exclusions were necessary to keep the project focused on deterministic, inspectable scheduling and version control semantics.

3.3 Success Criteria

The Evaluation Day and testing plan measured success criteria aligned with core objectives:

- **SC1 (Merge usability):** participants can resolve a planned merge conflict using the UI within a target time (goal: under 5 minutes for most users).
- **SC2 (Scheduling feasibility):** the auto-scheduler produces non-overlapping itineraries with non-zero travel gaps for a scenario with at least six stops across one to two days.
- **SC3 (Usability and accessibility):** the UI is understandable and usable for both Git-familiar and Git-novice users; critical accessibility issues are avoided and the system remains readable and navigable.

3.4 Non-functional Requirements

Non-functional requirements were also essential given external API dependencies and deployment:

- **Reliability:** graceful fallback when routing APIs fail; avoid corrupting repository state.
- **Performance:** responsive UI; reasonable planning time for typical stop counts (single-digit to low-teens).
- **Security and privacy:** session cookies; rate limiting; input validation; avoid collecting unnecessary personal data; GDPR awareness [**?**].
- **Deployability:** simple deployment model with persistent storage (SQLite) and environment variables for API keys.

3.5 Scope Boundary: Why AI Was Excluded from the MVP

The Project Specification explicitly excluded AI-powered itinerary generation from natural language prompts. This boundary served four practical purposes:

- **Determinism and explainability:** the scheduler must justify feasibility decisions (e.g., opening-hours nudges), which is easier with rule-based logic than with generative approaches.
- **Evaluation clarity:** success criteria focus on constraint satisfaction and merge usability, which require objective measurement.
- **Privacy and deployment:** avoiding model hosting and user preference profiling reduces privacy risks and operational complexity.
- **Time and risk control:** adding AI would broaden the problem into recommendation, preference learning, and natural-language understanding, which exceeds the MVP schedule.

Machine learning is still a credible future extension and is discussed in Chapter 9.

Chapter 4

Methodology

This chapter explains the development process, tool selection, testing methodology, and evaluation design used to deliver GiTrip.

4.1 Development Approach

GiTrip was developed using an iterative process combining:

- **Risk-driven prototyping:** early prototypes validated feasibility of snapshot storage, server-rendered UI, and routing integration.
- **Incremental feature delivery aligned to MoSCoW:** Must-haves were implemented first to reach an end-to-end MVP; Should-haves and hardening tasks followed.
- **Continuous feedback:** weekly supervisor meetings provided checkpoints for scope, presentation, and evaluation readiness.

This approach was chosen because the project's key risks were integration-heavy: routing APIs, interactive UI behaviour, and merge correctness under edge cases. In particular, the merge engine required three major rewrites as edge cases emerged during testing, which validated the decision to iterate early on high-risk components.

4.2 Choice of Tools and Technologies

GiTrip's major technology choices were guided by the requirement to deliver a complete, deployable web system with both server-rendered UI and asynchronous planning/merge operations.

4.2.1 Node.js and Express

Node.js provides a mature ecosystem for web servers, with excellent support for JSON-based data models [**?**]. Express was selected as a lightweight, widely-used framework that supports both HTML routes and JSON APIs cleanly [**?**]. This combination enables server-rendered views (for fast initial load and SEO-like discoverability) while still allowing interactive enhancements via client-side JavaScript.

4.2.2 SQLite for Persistent Storage

SQLite was chosen for persistent storage because it is reliable, lightweight, and appropriate for a project deployed as a single web service [**?**]. Its file-based deployment model aligns with container deployment and makes backups simple. While SQLite is not a distributed database, GiTrip's collaboration model (branch-and-merge) does not require multi-writer database replication; concurrency is managed at the application level.

4.2.3 Mapping and Geocoding

Leaflet was used for the interactive map because it is a well-established open-source mapping library that integrates cleanly with OpenStreetMap tiles [**?**, **?**]. Nominatim was used for place lookup and geocoding due to its OpenStreetMap integration and accessible API [**?**]. A server-side proxy route is used to apply rate limiting and caching to avoid overloading the upstream service.

4.2.4 Routing Providers

OpenRouteService was used for travel-time matrix routing for non-transit modes, which is important for ordering and scheduling decisions [**?**]. Google Directions API was used for public transit legs, because transit routing requires schedule-aware directions and detailed steps for UI explanation [**?**]. A key design requirement was provider fallback: when API calls fail, GiTrip can degrade gracefully using Haversine-based estimates and constant-gap defaults.

4.2.5 Usability Frameworks

Nielsen's usability principles were used as a practical heuristic framework for iterative UI refinement [**?**]. WCAG 2.2 was used to guide accessibility checks and avoid critical issues [**?**]. For evaluation design, SUS and TAM-inspired constructs were used to capture perceived usability and acceptance [**?**].

4.3 Testing Strategy

Testing was treated as a first-class requirement due to the complexity of merge logic and scheduling constraints. The strategy combined:

- **Unit tests** for opening-hours parsing, time utilities, and merge conflict detection.
- **Functional tests** covering end-to-end workflows (plan, commit, branch, merge, resolve, export).
- **Non-functional checks** covering security headers, rate limiting, and accessibility considerations.

All unit tests use the Node.js native test framework (`node:test`) with strict assertions.

4.4 Evaluation Design

Evaluation was conducted via:

- **User Acceptance Testing (UAT)** on Evaluation Day with an observer checklist recording objective task completion, time-to-finish, and hints given.
- **A post-demo survey** (Microsoft Forms) collecting Likert responses (including SUS), Git familiarity, and free-text feedback.

This design intentionally combines objective evidence (task completion and time) with subjective user perception (usability and perceived value), aligning with a pragmatic interpretation of TAM in small-sample contexts.

4.5 Ethics and Data Handling

The evaluation collected minimal personal data. Survey responses were anonymous, and the system does not require real personal identity information for most features. Where authentication is used, sessions are stored via random identifiers and protected by HttpOnly cookies. GDPR considerations are discussed in Chapter 5 [**?**].

Chapter 5

System Design

This chapter describes GiTrip's architecture, data model, and major subsystems: repository semantics, planner, routing, merging, UI interaction surfaces, and supporting features. Each section explains not only what was built but why that approach was chosen over alternatives.

5.1 High-level Architecture

GiTrip is a server-rendered web application with an Express backend and EJS templates for UI routes, complemented by JSON API endpoints for asynchronous operations. UI routes under `/ui/...` render pages such as the repo view, planner, merge UI, and explore mode. API routes under `/api/...` handle operations including commits, branches, merges, routing geometry, and quick route optimisation. Persistent state is stored in a SQLite database containing repositories, branches, commits, and social/collaboration metadata.

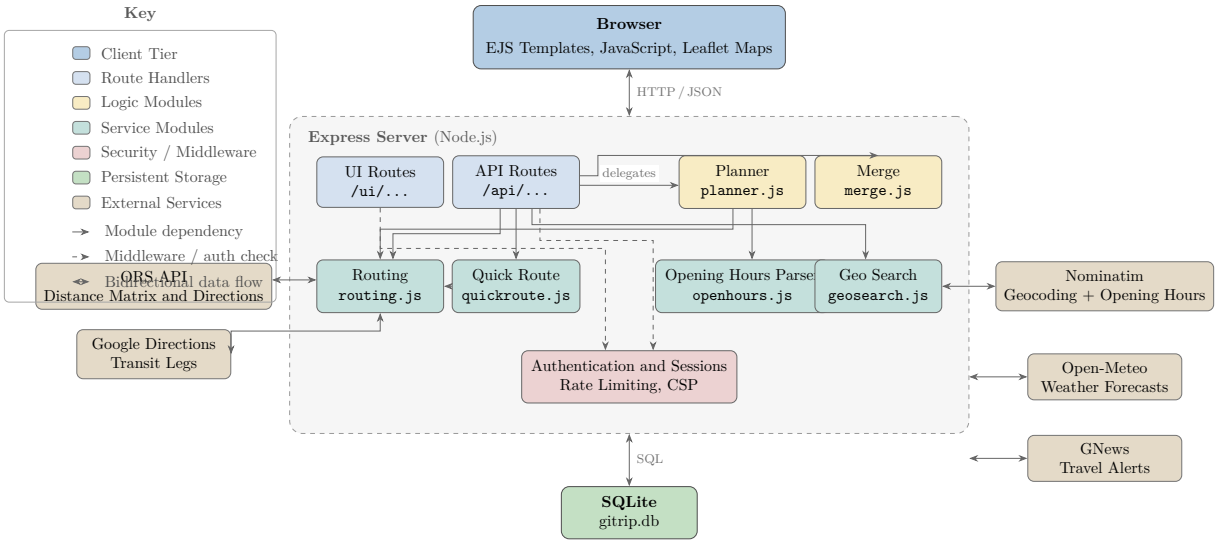


Figure 5.1: GiTrip high-level architecture. The browser communicates with the Express server over HTTP/JSON. *UI Routes* render pages while *API Routes* delegate to the *Planner*, *Merge*, *Routing*, and *Geo Search* modules. *Quick Route* delegates to *Routing*, which in turn calls ORS for non-transit modes and Google Directions for transit legs. The *Planner* consults the *Opening Hours Parser*, a local utility that validates time intervals against OSM opening-hours strings supplied by Nominatim via *Geo Search*. Authentication middleware enforces session management, rate-limiting, and Content Security Policy headers across all routes. Server route handlers fetch weather forecasts from Open-Meteo and travel alerts from GNews. All persistent state is stored in a SQLite database.

5.2 Data Model and Repository Semantics

GiTrip's database schema includes the following core tables:

- **repos:** repository metadata (title, owner, visibility, current branch, fork source).
- **branches:** named pointers to commits (head commit).
- **commits:** commit metadata and full snapshot storage.

Additional tables support authentication (**users**, **sessions**) and social/collaboration (**repo_collaborators**, **repo_stars**). Supporting planning features are stored in **travel_alerts** and **packing_checklists**. Database indexes are defined on all foreign-key columns (e.g., `commits.repo_id`, `branches.repo_id`, `repo_stars.repo_id`) to ensure query performance scales with data growth.

Figure 5.2 shows the entity relationship diagram for GiTrip's data model. The core version-control triad (Repo, Branch, Commit) mirrors Git semantics, while the embedded Snapshot structure captures itinerary-specific data as JSON within each commit row.

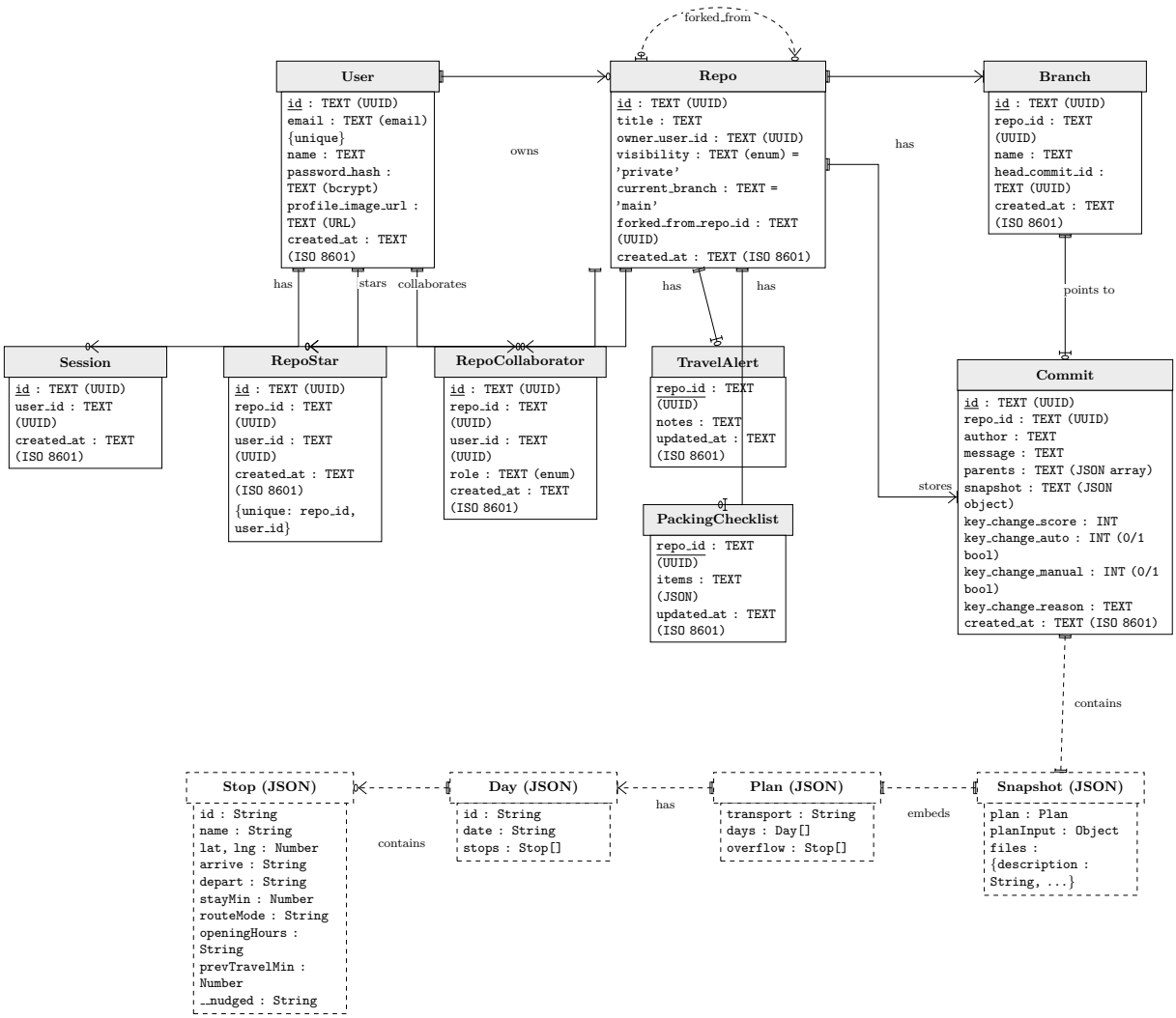


Figure 5.2: Entity relationship diagram for GiTrip's data model using Crow's foot notation. Primary keys are underlined. Solid-bordered entities are persisted as SQLite tables; dashed-bordered entities represent JSON structures embedded within the `snapshot` column of each Commit. Crow's foot endpoints: double bar = exactly one, circle–bar = zero or one, fork–bar = one or many, fork–circle = zero or many.

5.2.1 Snapshot Storage

Each commit stores a complete snapshot rather than a delta. This aligns with Git's conceptual model of snapshots [**?**]. In GiTrip, the snapshot contains:

- `snapshot.plan`: the itinerary (days and stops).
- `snapshot.planInput`: planner inputs for reproducibility and UI defaults.
- `snapshot.files`: a key-value map for branch-level metadata. Currently stores the trip description (`files.description`), which collaborators can edit independently on separate branches. The merge engine performs per-key three-way merging on this map, detecting conflicts when both sides change the same key.

Snapshot storage simplifies branch switching and merging at the cost of increased storage size.

This trade-off is appropriate for itinerary data, which is small compared to typical binary files in software repositories.

5.2.2 Plan Schema

A plan includes:

- `transport` mode (driving/walking/cycling/transit),
- `days[]` each with `id`, `date`, and `stops[]`,
- each stop includes identity, location, scheduling, and routing metadata.

Stops also carry planner constraint fields such as strict ordering, time windows, fixed-time flags, and opening-hours strings. A `__nudged` marker records explainable adjustments made by the scheduler (e.g., nudged due to opening hours).

5.2.3 Module Structure

Figure 5.3 shows the UML class diagram for GiTrip's server-side modules. Each module is represented as a class with its exported functions as public operations (`+`) and internal helper functions as private operations (`-`). The `Server` module acts as the central orchestrator, delegating to specialised modules for planning, merging, routing, and geocoding. `TTLCache` is the only formal JavaScript class in the codebase; all other modules use the CommonJS `module.exports` pattern.

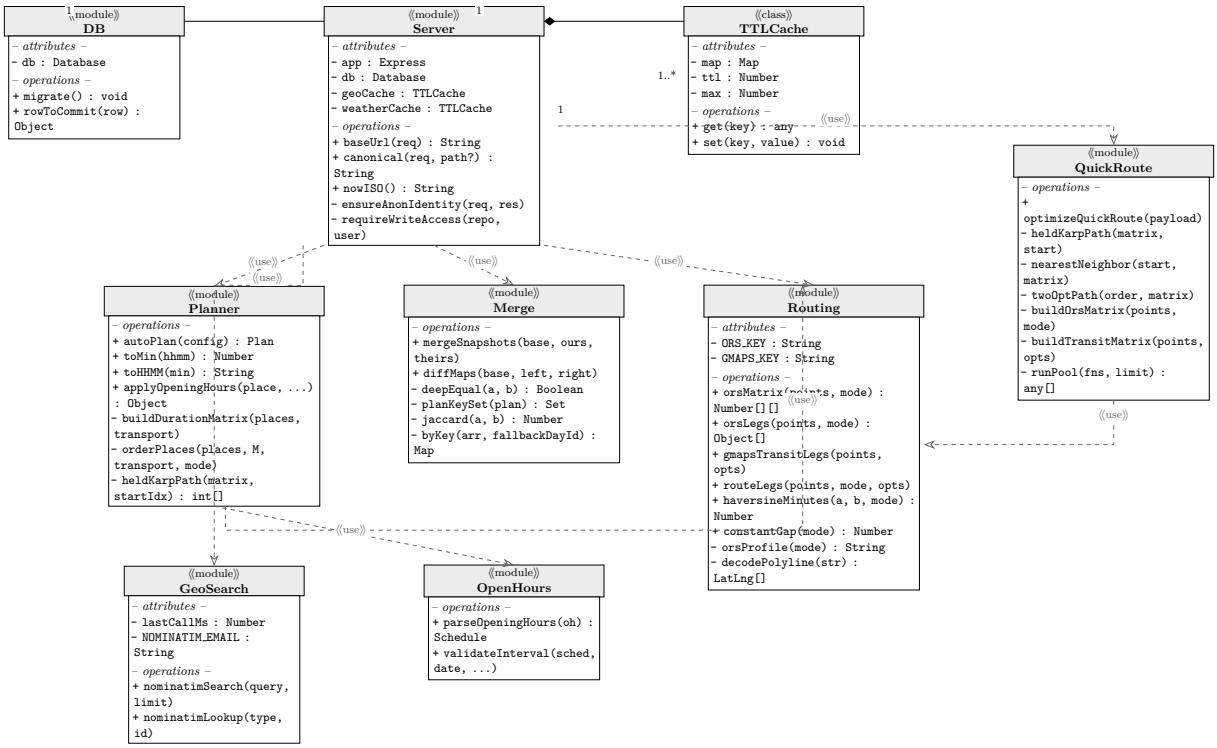


Figure 5.3: UML class diagram for GiTrip's server-side modules. Each module is shown with public (+) and private (-) operations. Dashed open arrows denote ⟨⟨use⟩⟩ dependencies; the filled diamond denotes composition (Server owns TTLCache instances). `TTLCache` is a formal JavaScript class; all other boxes are CommonJS modules.

5.3 Routing and Geocoding Design

5.3.1 Place Search

GiTrip proxies Nominatim search/lookup via server endpoints to enable caching and rate limiting [**?**]. Results are re-ranked using approximate string matching to prioritise substring matches and reduce diacritic/punctuation sensitivity [**?**].

5.3.2 Routing Providers and Fallback

For non-transit modes, GiTrip requests a travel-time matrix from OpenRouteService [**?**]. For transit, GiTrip uses Google Directions API for sequential legs [**?**]. When providers fail or are unavailable, GiTrip falls back to:

- Haversine-based time estimates for ordering,
- constant default travel gaps for safety.

This design ensures the planner remains usable even in degraded conditions, which is important for evaluation environments and offline/limited-connectivity contexts.

5.4 Auto-scheduler Design

The scheduler must construct a feasible plan that respects constraints. GiTrip's scheduler is designed around four stages:

1. **Matrix construction:** request pairwise travel durations (when available).
2. **Ordering:** determine visit order using strict order constraints when provided; otherwise apply Nearest Neighbour ordering as a heuristic baseline [**?**].
3. **Packing into days:** greedily place stops into day windows while maintaining non-overlap and travel gaps.
4. **Constraint enforcement:** apply opening hours, desired windows, and fixed-time constraints; if a stop cannot fit, spill it to a subsequent day or overflow.

5.4.1 Opening Hours Nudging

Opening hours are treated as a hard-ish constraint: if the proposed interval falls in a closed period, the scheduler nudges to the next open slot where feasible; otherwise the stop is rejected for that day. This is crucial for feasibility and directly addresses common itinerary failures in real planning.

5.4.2 Compact vs Sparse Modes

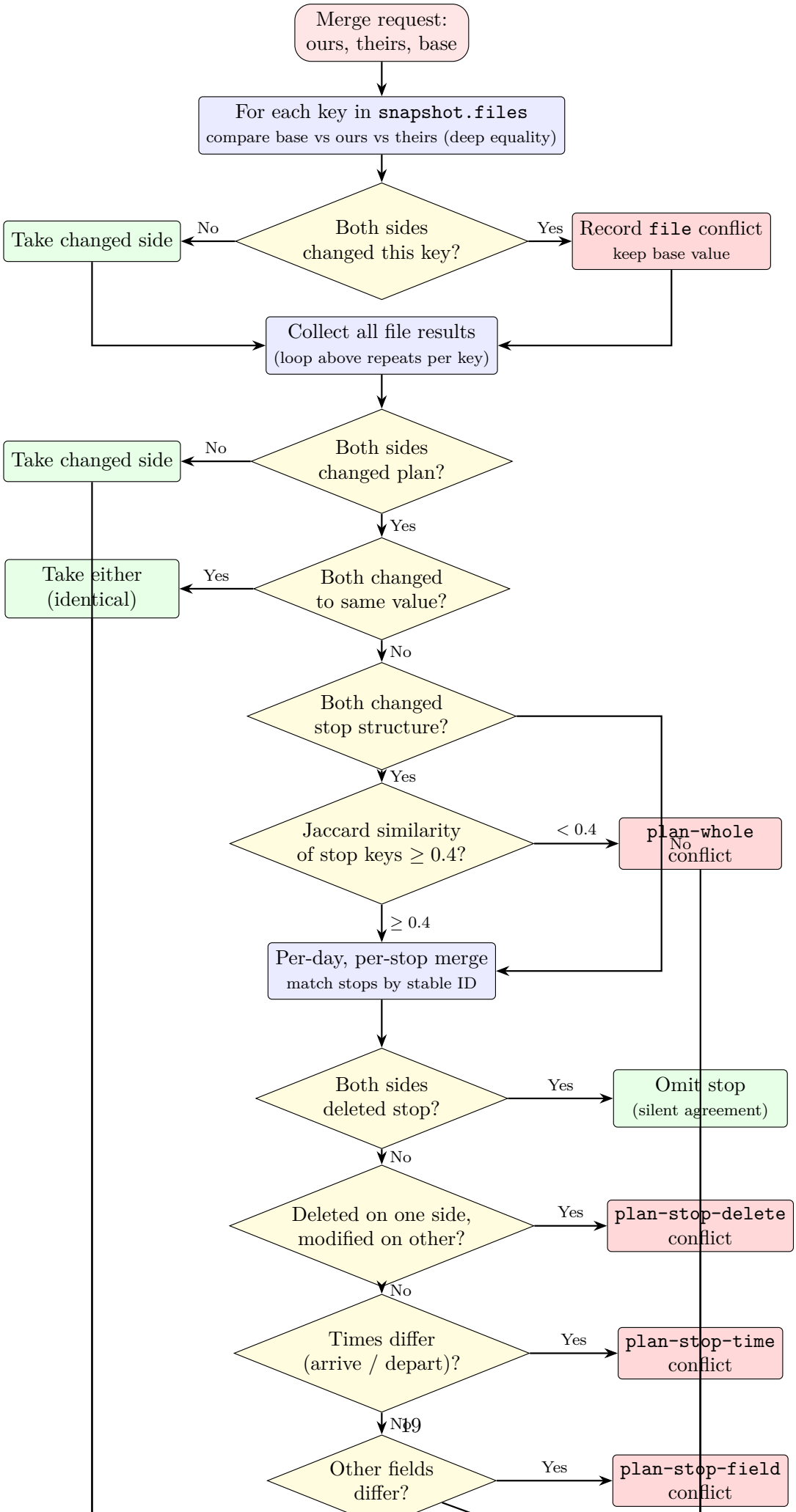
To improve user control over schedule density, the scheduler supports "compact" and "sparse" modes. Compact mode packs stops tightly, while sparse mode distributes stops across the active window by inserting additional gap time. This preference improves perceived usefulness because different trip styles (e.g., city sprint vs relaxed exploration) require different pacing.

5.5 Merge Design

GiTrip implements a Git-style three-way merge for snapshots. The merge process uses:

- a **base** snapshot (lowest common ancestor),
- **ours** and **theirs** snapshots (branch heads),
- a merge engine producing a merged snapshot plus a list of conflicts.

Figure 5.4 illustrates the decision logic of the merge engine.


Merge request: ours, theirs, base
For each key in snapshot.files
compare base vs ours vs theirs (deep equality)
Both sides changed this key?
No
Take changed side
Yes
Record file conflict
keep base value
Collect all file results
(loop above repeats per key)
Both sides changed plan?
No
Take changed side
Yes
Both changed to same value?
Yes
Take either (identical)
No
Both changed stop structure?
Yes
Jaccard similarity of stop keys ≥ 0.4?
< 0.4
plan-whole
No conflict
≥ 0.4
Per-day, per-stop merge
match stops by stable ID
Both sides deleted stop?
Yes
Omit stop
(silent agreement)
No
Deleted on one side, modified on other?
Yes
plan-stop-delete
conflict
No
Times differ (arrive / depart)?
Yes
plan-stop-time
conflict
No
Other fields differ?
Yes
plan-stop-field
conflict

5.5.1 Equality and Stability

Early merge implementations that relied on `JSON.stringify` can be brittle because key insertion order may differ and `undefined` values are dropped. The final implementation uses deep structural equality, making comparisons key-order independent.

Additionally, merged ordering is stabilised by sorting days by ISO date and sorting stops by arrival time (then name). This reduces surprising reordering after merges.

5.5.2 Conflict Types

Conflicts surfaced include:

- **file:** both sides changed the same key in `snapshot.files` (e.g. the trip description).
- **plan-stop-time:** both sides changed arrival/departure times differently for the same stop.
- **plan-stop-delete:** one side deleted a stop while the other modified it.
- **plan-stop-field:** times agree but both sides changed selected fields differently (e.g., name, lat/lng, notes, stayMin, routeMode).
- **plan-whole:** plans diverged heavily (Jaccard similarity of stop-key sets below 0.4), requiring whole-plan selection.

This set is intentionally constrained to keep the UI manageable for non-experts while still preventing silent data loss.

5.6 User Interface Design

GiTrip uses server-rendered pages enhanced by client-side JavaScript. Two interactive surfaces support editing:

- **Timeline editor:** draggable blocks and handles adjust arrival/departure times with snapping and constraint clamping.
- **Map editor:** draggable numbered markers adjust stop coordinates and update route geometry.

5.6.1 Easy Mode vs Git Mode

Survey feedback during development indicated Git concepts are confusing for non-technical users. GiTrip therefore includes a mode toggle:

- **Easy mode:** hides branches/commits/merge; offers “Save trip” with non-fast-forward friendly behaviour.
- **Git mode:** exposes full repository semantics and merge UI.

This design supports two user personas without splitting the product into separate applications.

5.7 Security, Privacy, and Accessibility Design

GiTrip includes several practical web security measures:

- session cookies are HttpOnly and SameSite=Lax,
- security headers include CSP, frame blocking, and HSTS in production,
- rate limiting is applied globally and more strictly on sensitive endpoints (four tiers: global, API, geo search, and authentication),
- prepared statements are used for SQLite queries to reduce SQL injection risk,
- error responses return generic messages to users while logging full details server-side, preventing information leakage of internal paths or stack traces.

Accessibility is guided by WCAG 2.2 [**?**]. The UI uses labelled controls and avoids information conveyed only by colour; however, interactive map/timeline controls remain a known accessibility challenge and are discussed as a limitation.

Chapter 6

Implementation

This chapter documents how the design was realised in code, covering major implementation modules and the reasoning behind key decisions.

6.1 Backend Structure

The backend is a Node.js + Express application [**?**, **?**]. The main server file configures:

- EJS templating for UI routes,
- JSON body limits for planner payloads,
- authentication sessions (`gitrip_sid`),
- security headers and rate limiting,
- caching for external API calls (geocoding and weather).

6.1.1 Database Layer

The database layer uses `better-sqlite3` and maintains schema migrations in a single module. Commit rows store JSON fields (`parents`, `snapshot`) as strings and are deserialised on load. The schema intentionally does not enforce foreign keys at the SQLite level; relationships are handled logically by the application. Indexes are created on all foreign-key columns (e.g., `commits.repo_id`, `branches.repo_id`, `repo_stars.repo_id/user_id`) to maintain query performance as the number of repositories and commits grows.

6.1.2 Error Handling

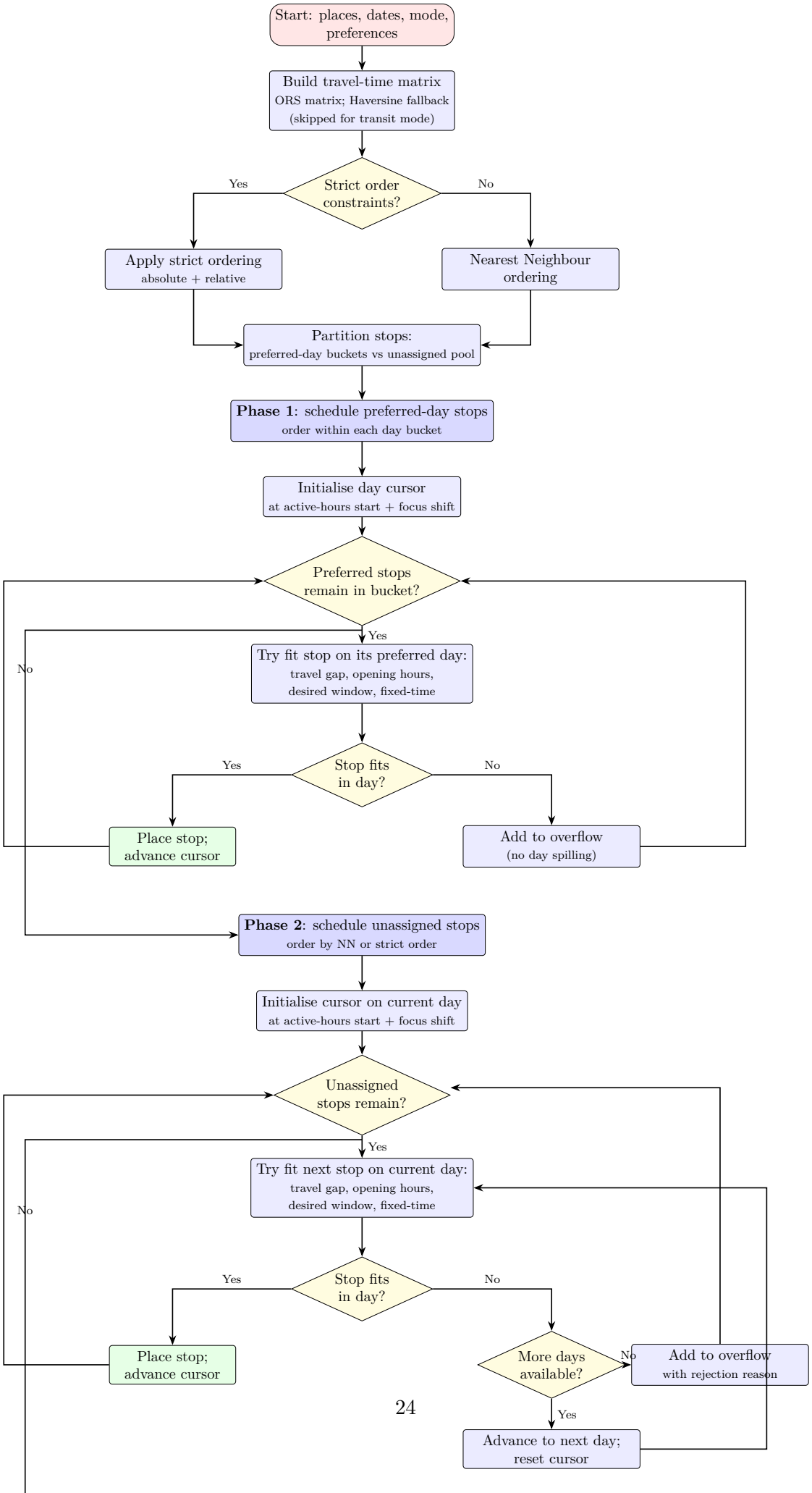
All error responses sent to users use generic messages (e.g., "Planner error. Please try again.") rather than exposing internal error details. Full error information is logged server-side via `console.error` for debugging. This prevents information leakage of internal paths, stack traces, or dependency details that could aid reconnaissance.

6.2 Planner Implementation

The auto-scheduler is implemented in `server/planner.js`. Key components include:

- **Time helpers:** `toMin` and `toHHMM`.
- **Matrix building:** ORS matrix for non-transit; null for transit.
- **Ordering logic:** strict order handling (absolute or relative), plus Nearest Neighbour when no strict orders exist.
- **Constraint fit check:** `applyOpeningHours` integrates active hours, desired windows, fixed-time behaviour, and opening-hours nudging.
- **Packing loop:** schedules preferred-day buckets first, then unassigned stops, spilling to later days or overflow when necessary.

The planner records travel minutes (`prevTravelMin`) and effective route mode per leg, including detection of transit vs walking in cases where the transit provider returns walking-only legs.


Start: places, dates, mode, preferences
Build travel-time matrix
ORS matrix; Haversine fallback
(skipped for transit mode)
Strict order constraints?
Yes
No
Apply strict ordering
absolute + relative
Nearest Neighbour ordering
Partition stops:
preferred-day buckets vs unassigned pool
Phase 1: schedule preferred-day stops
order within each day bucket
Initialise day cursor
at active-hours start + focus shift
Preferred stops remain in bucket?
Yes
No
Try fit stop on its preferred day:
travel gap, opening hours,
desired window, fixed-time
Stop fits in day?
Yes
No
Place stop; advance cursor
Add to overflow
(no day spilling)
Phase 2: schedule unassigned stops
order by NN or strict order
Initialise cursor on current day
at active-hours start + focus shift
Unassigned stops remain?
Yes
No
Try fit next stop on current day:
travel gap, opening hours,
desired window, fixed-time
Stop fits in day?
Yes
No
Place stop; advance cursor
More days available?
No
Add to overflow
with rejection reason
Yes
Advance to next day; reset cursor

6.3 Merge Implementation

The merge engine is implemented in `server/merge.js`. Improvements made after the Progress Report address known risks:

- **Deep structural equality:** avoids `JSON.stringify` brittleness.
- **Robust stop identifiers:** if stops lack IDs, stable IDs are generated from name + coordinates + day context, reducing merge drop-outs.
- **Deterministic ordering:** days sorted by date; stops sorted by arrival time.
- **Deletion semantics:** explicit `plan-stop-delete` conflict for delete-vs-modify.
- **Field-level conflicts:** six selected fields (`name`, `lat`, `lng`, `notes`, `stayMin`, `routeMode`) produce `plan-stop-field` conflicts when both sides diverge.

Structural divergence is detected using a Jaccard similarity heuristic over stop-key sets; if the Jaccard index falls below 0.4, a `plan-whole` conflict is produced and the UI requires choosing an entire plan. Without this guard, per-stop merging would attempt to align two largely unrelated itineraries—for example, one collaborator replacing the plan with stops in Tokyo while the other replaces it with stops in Paris—producing a nonsensical hybrid plan with no meaningful correspondence between stops.

6.4 Routing Implementation

Routing integration is implemented in `server/routing.js`:

- ORS matrix requests use the matrix endpoint and return durations in minutes [**?**].
- ORS directions requests fetch per-leg geometry for map rendering.
- Google Directions transit routes provide detailed per-step segments, including polylines decoded via the standard polyline algorithm [**?**].
- Fallback estimates use Haversine distance.

6.5 Quick Explore Mode

Quick explore mode provides a Google-Maps-like workflow where users quickly enter destinations, optimise an order, and optionally start a repository from the result. This is implemented via `server/quickroute.js`:

- for small instances ($N \leq 12$), an exact Held-Karp dynamic programming solver is used [**?**, **?**];
- for larger instances, NN is used with 2-opt refinement.

The quick route engine builds a travel-time matrix using ORS, transit pairwise calls for small transit sets (limited to $N \leq 7$ to control API cost), or Haversine fallback. Concurrency is bounded to a pool of four simultaneous requests to reduce API burst risk.

6.6 Algorithm Visualisation

GiTrip includes an educational algorithm visualisation page (`server/public/algorithm.js`) that demonstrates three route-ordering algorithms with step-by-step animation on a Leaflet map:

1. **Nearest Neighbour (NN):** greedy selection of the closest unvisited stop; $O(N^2)$ time.
2. **NN + 2-opt:** starts from an NN solution and iteratively reverses sub-segments to shorten the route; used for instances with more than 12 stops.
3. **Held-Karp dynamic programming:** exact bitmask DP solver in $O(2^N \cdot N^2)$ time; guarantees optimality but is limited to N $\leq$ 12 stops due to exponential growth.

The visualisation fetches real routing geometry from the server (`/api/algo-matrix` and `/api/algo-geometry`), supports multiple transport modes, and includes playback controls (play, pause, step, reset, speed slider), synchronised pseudocode highlighting, animated scan lines showing which edges are being evaluated, and a cost comparison panel.

6.7 Frontend Implementation

GiTrip's pages are server-rendered with EJS templates and enhanced with client-side JavaScript:

- **Repo UI:** displays trip plan, map, timeline, branches, commits, settings, and exports.
- **Merge UI:** presents conflicts and resolution controls, then submits a resolution payload to create a merge commit.
- **Planner UI:** collects inputs and submits to the planner endpoint to generate a new commit on a planner branch.

6.7.1 Timeline and Map Editing

The timeline editor uses draggable SVG elements with snapping to 5-minute increments. It clamps edits based on travel constraints:

- a stop cannot start before the previous stop's departure plus travel time,
- a stop cannot extend beyond the next stop's arrival,
- minimum stop duration is enforced.

The map editor uses Leaflet markers with numbered pins [**?**]. In edit mode, markers are draggable and route polylines are updated. Transit legs display additional stop markers and per-leg mode styling (e.g., dashed walking segments).

6.8 Exports and PWA Features

6.8.1 PDF Export

PDF export is implemented using browser printing (`window.print()`), which is robust and avoids server-side rendering complexity.

6.8.2 iCalendar Export

GiTrip generates `text/calendar` content in iCalendar format and serves it as an `.ics` download. The iCalendar format is defined by RFC 5545 [**?**].

6.8.3 PWA and Offline

GiTrip includes PWA support through a web application manifest and a service worker. The manifest standard is specified by W3C [**?**], and service workers are defined by W3C as an event-driven background worker model enabling offline caching [**?**]. Offline behaviour focuses on caching core assets and recently accessed pages; limitations include the inability to guarantee offline routing responses.

6.9 Supporting Features

6.9.1 Weather Integration

Each day in a plan displays a weather forecast retrieved from the Open-Meteo API. The server fetches daily forecasts (temperature range, precipitation probability, and weather code) using the first stop's coordinates for each day, with a 30-minute TTL cache to reduce external requests. Historical data is fetched for past dates and forecast data for future dates.

6.9.2 Travel Alerts

GiTrip retrieves travel disruption news via the GNews API. City names are extracted from stop names in the plan, and a search query combining those cities with disruption keywords (strike, closure, protest, etc.) is issued. Because articles about upcoming disruptions are typically published before the affected dates, the search window extends from seven days before the first trip date through the last trip date, and each article is assigned to the nearest matching trip date. Results are displayed per day with badge counts. Users can also add manual alert notes stored in the `travel_alerts` table.

6.9.3 Public Gallery with Pagination

The public gallery (`/ui/public`) displays trips that owners have marked as public. The listing supports sorting by recency or star count and is paginated at 12 trips per page using SQL `LIMIT/OFFSET` queries, preventing unbounded response sizes as the number of public trips grows.

6.10 Deployment and Operations

GiTrip supports deployment via containerisation with a persistent volume for the SQLite database file (`gitrip.db`). The server starts on `PORT` (default 4000) and automatically increments if the port is in use. Environment variables provide routing API keys. The system was deployed for Evaluation Day to allow participants to access the application from multiple machines and personal devices.

Chapter 7

Testing

This chapter provides evidence that GiTrip was tested systematically at unit, functional, and non-functional levels. The goal is to demonstrate correctness for core algorithms (merge and scheduler) and reliability for user-facing workflows.

7.1 Unit Testing

GiTrip includes five unit test files with 64 test cases total, executed using the Node.js native test framework (`node:test`) with strict assertions.

Table 7.1: Summary of unit test coverage (64 test cases).

Test file	**Primary coverage**	**# tests**
`openhours.test.js`	Opening-hours parsing and interval validation	11
`merge.test.js`	Three-way merge and conflict detection	16
`planner.test.js`	Time utilities and opening-hours scheduling	13
`routing.test.js`	Haversine fallback and constant-gap defaults	11
`api-keys.test.js`	External API connectivity (ORS, Google, GNews, Open-Meteo, Nominatim)	13
Total		**64**

7.1.1 Opening Hours Parsing and Validation

The opening-hours module tests include:

- standard weekday ranges (e.g., `Mo-Fr 09:00-18:00`),
- wrap-around ranges (e.g., `Fr-Mo`),
- `off` keyword meaning closed,
- multi-span days with breaks,
- compound rules across different days,
- empty/null input handling.

Validation tests check whether a proposed interval is open, closed, nudged to a next open slot, too long to fit, or exactly fits a boundary.

7.1.2 Merge Conflict Detection

Merge tests cover:

- clean merges where only one side changes,
- deep-equality behaviour that is independent of key order,
- deletions (delete-vs-modify yields explicit `plan-stop-delete` conflict),
- time conflicts (`plan-stop-time`),
- field conflicts (`plan-stop-field`),
- whole-plan conflicts when similarity is low (`plan-whole`),
- deterministic ordering of merged days and stops.

7.1.3 Planner Time Utilities and Scheduling Logic

Planner tests cover both time-conversion functions (`toMin`, `toHHMM`, including null/empty inputs, negative clamping, and fractional rounding) and the `applyOpeningHours` constraint engine, including:

- unconstrained scheduling (no constraints; starts at proposed arrival),
- clamping to active period,
- soft desired windows (`desiredStart`, `desiredEnd`),
- opening-hours nudging to the next open slot,
- hard fixed-time windows (failure when travel arrives too late),
- rejection when the stop does not fit within the active window,
- rejection when the venue is closed all day.

7.1.4 Routing Fallback Logic

Routing tests cover the Haversine fallback estimator and constant-gap defaults used when external routing APIs are unavailable:

- identity case (same point returns zero travel time),
- realistic distance estimation (London to Paris within expected range),
- mode ordering (walking slower than cycling, cycling slower than driving),
- graceful handling of null or missing coordinates,
- constant-gap values for each transport mode.

7.1.5 External API Connectivity

API connectivity tests verify that every third-party service required by GiTrip is correctly configured and reachable at test time:

- **Presence checks** – each required environment variable (`ORS_API_KEY`, `GOOGLE_DIRECTIONS_API_KEY`, `NEWS_API_KEY`) is set in `.env`.
- **ORS (OpenRouteService)** – a real matrix request and a directions request confirm the key is accepted and returns valid duration data.
- **Google Directions** – both driving and transit modes are tested; the test asserts that the response status is `OK` (not `REQUEST_DENIED`), catching billing or API-enablement issues early.
- **GNews** – a search query confirms the key returns articles and a travel-specific query returns relevant results.
- **Open-Meteo** – both the forecast and archive endpoints are tested with a London coordinate, verifying that daily temperature and precipitation data are returned in the expected format.
- **Nominatim** – a geocoding search confirms the endpoint returns results with latitude, longitude, and display name. A second test verifies that the `extratags` field (which carries opening-hours data from OSM) is present when requested.

These tests act as a deployment smoke test: a single failing assertion pinpoints exactly which external service is misconfigured, rather than requiring manual investigation of silent routing fallbacks.

7.2 Functional Testing

Functional testing verified that core workflows operate end-to-end. Table 7.2 summarises key cases.

Table 7.2: Functional test cases for core workflows (summary).

Workflow	Result	Evidence
Create repo; view/edit tabs render	Pass	Manual verification
Run auto-planner; commit created; redirect	Pass	Commit history updated
Timeline edit; save commit; reload matches	Pass	Snapshot persists
Map marker drag; recompute route; save	Pass	Geometry updates
Create branch; switch branch; plan changes	Pass	Head snapshot changes
Merge branches with time conflict; resolve	Pass	Conflict UI + merge commit
Quick explore optimise; start repo from route	Pass	Repo created from route
Export PDF view	Pass	Browser print output
Export iCalendar (ICS)	Pass	Calendar import works

7.3 Non-functional Testing and Checks

7.3.1 Security Checks

GiTrip includes:

- Content Security Policy headers and anti-framing headers,
- four-tier rate limiting (global, API, geo search, and authentication endpoints),
- prepared statements for all database access,
- conservative input normalisation for user-provided fields,
- generic error messages in all user-facing responses (internal details logged server-side only, preventing information leakage).

These checks are not a substitute for a full penetration test, but they provide reasonable baseline hardening for a deployed student project.

7.3.2 Accessibility Checks

Accessibility was guided by WCAG 2.2 [**?**]. Checks included:

- consistent labelling of controls,
- avoiding information conveyed only via colour,
- readable typography and spacing,
- basic keyboard navigation for non-map interactions.

A more formal audit using automated tools such as Lighthouse is recommended for future work [**?**]. The interactive map and timeline remain difficult to fully validate for accessibility within the project scope.

7.3.3 Resilience to External API Failures

Functional tests included scenarios where routing keys were missing or external calls failed. In these cases, GiTrip falls back to approximate travel-time estimates and still produces a schedule (potentially with reduced accuracy). This behaviour supports usability in constrained environments.

7.4 Testing Summary

The testing evidence demonstrates that the most error-prone components (merge logic and scheduler constraints) are covered by automated unit tests and that major user workflows function end-to-end. Remaining limitations are largely usability and accessibility challenges inherent in interactive planning UIs, rather than basic correctness failures.

Chapter 8

Evaluation

This chapter reports the evaluation conducted on Evaluation Day (Wednesday 25th February 2026) and via a follow-up survey. It presents both objective user acceptance test results and subjective survey findings, and interprets them against the project's success criteria.

8.1 Evaluation Day Setup

Evaluation Day took place in the Department of Computer Science, Room CS0.03, Table B01, slot 13:05–13:45, with the project titled "GiTrip — Git for Collaborative Trip Planning". The demonstration environment used four Linux machines plus the author's laptop so that multiple participants could test branching and merging behaviours in parallel. Additionally, the application was deployed publicly so attendees could also use their own laptops and devices.

8.2 Participants

Two data sources were collected:

- **User Acceptance Test (UAT):** 10 in-person participants (P1–P10) with observer-recorded results.
- **Survey responses:** 13 responses collected via Microsoft Forms (including remote responses). The survey reported an average completion time of 08:46 minutes.

Survey respondents had mixed Git familiarity: 2 not familiar, 5 somewhat familiar, 3 comfortable, and 3 very comfortable.

8.3 User Acceptance Testing Tasks

Three tasks were measured using an observer checklist to provide objective evidence.

8.3.1 Task T1: Auto-schedule Feasibility

Scenario: at least six stops across one to two days. Evidence recorded: completion, time-to-finish, non-overlap, and non-zero travel gaps.

Table 8.1: UAT Task T1 — Auto-schedule feasibility.

Participant	Completed?	Time	Non-overlapping?	Non-zero travel gaps?
P1	Y	3:12	Y	Y
P2	Y	4:45	Y	Y
P3	Y	2:58	Y	Y
P4	Y	5:30	Y	Y
P5	Y	3:44	Y	Y
P6	Y	6:02	Y	Y
P7	N	8:00*	–	–
P8	Y	4:11	Y	Y
P9	Y	3:55	Y	Y
P10	Y	4:28	Y	Y

*P7 timed out due to difficulty locating the auto-scheduler button without a hint.

Across successful completions (9/10), the mean completion time was 4:18 (median 4:11), and all successful outputs were non-overlapping with non-zero travel gaps. This supports Success Criterion SC2.

8.3.2 Task T2: Merge Conflict Resolution

Scenario: pre-seeded repository with two branches producing at least one timing conflict. Evidence recorded: resolved or not, time-to-resolve, and number of hints given.

Table 8.2: UAT Task T2 — Merge conflict resolution.

Participant	Resolved?	Time	Hints given
P1	Y	2:34	0
P2	Y	4:50	1
P3	Y	1:58	0
P4	Y	3:22	0
P5	Y	4:47	1
P6	N	6:10*	2
P7	Y	3:05	1
P8	Y	2:41	0
P9	Y	4:33	1
P10	Y	3:18	0

*P6 selected the wrong branch and ran out of time.

Overall, 9/10 participants resolved the conflict; 9/10 achieved a resolution within 5 minutes. The mean time across all participants was 3:44 (mean hints 0.6). This supports Success Criterion SC1.

8.3.3 Task T3: Quick Edit + Save Without Git Explanation

Task T3 measured whether a participant could make a small timeline/map edit and save without requiring Git vocabulary explanation.

Table 8.3: UAT Task T3 — Quick edit and save without Git explanation.

Participant	Completed?	Notes
P1	Y	Edited timeline stop duration naturally
P2	Y	Used "Save" without prompting
P3	Y	–
P4	N	Asked "does saving here create a version?"
P5	Y	–
P6	Y	–
P7	Y	–
P8	N	Hesitated; unsure if edit was auto-saved
P9	Y	–
P10	Y	–

8/10 completed without needing Git explanation. This supports the value of Easy mode and indicates that versioning concepts can still raise questions (notably around mental models of saving).

8.4 Survey Results

The Evaluation Day survey collected 13 responses. Key quantitative findings included:

- Git familiarity was mixed: 15% not familiar (2/13), 38% somewhat familiar (5/13), 23% comfortable (3/13), 23% very comfortable (3/13).
- Participants tried a range of features: 11 created/opened a repository, 9 ran the auto-scheduler, 5 attempted branch/compare and merge/conflict resolution, and 4 used quick route mode.

Likert responses across Core Functionality and Overall Impression were strongly positive, with responses concentrated in Agree/Strongly Agree. The Git Concept Understanding section was also positive but showed more neutral responses, suggesting that the conceptual model is understandable but still benefits from guidance for some users.

The survey included the System Usability Scale (SUS). Due to the small sample size and the visual aggregation format, this report interprets SUS qualitatively rather than claiming a statistically robust SUS score. The overall pattern indicates that positive usability statements were generally endorsed while negative statements about complexity and inconsistency were generally rejected.

8.4.1 Qualitative Feedback

Free-text responses provide additional evidence of perceived value and pain points. Representative themes include:

Most useful/impressive:

- Auto-scheduler usefulness and edge-case handling (e.g., clearly showing unscheduled stops with reasons).
- Novelty of Git-style planning and merge conflict resolution in a non-code context.
- Educational value of algorithm visualisation.

Most frustrating:

- Some users perceived Git concepts as requiring learning overhead.
- Merge conflicts can be confusing without guidance (consistent with UAT hint patterns).
- A routing limitation was reported for special transport modes (e.g., ferries), reflecting dependency on external routing providers.

Would users adopt GiTrip? Most respondents indicated "yes" or "maybe", with collaboration and "keeping everything in one place" cited as major benefits. The strongest blockers were perceived complexity for casual trips and routing-provider constraints for niche transport types.

8.5 Discussion Against Success Criteria

Overall, the evaluation provides strong evidence that core objectives were achieved:

- The auto-scheduler reliably produced feasible plans for typical scenarios (SC2 met).
- Merge conflict resolution was achievable by most participants within the target time (SC1 met).
- Usability feedback validates the Easy mode approach while highlighting discoverability and conceptual learning as remaining improvement areas (SC3 partially met, with known limitations).

8.6 Threats to Validity

The evaluation has limitations typical of student project evaluations:

- **Sample bias:** many participants were likely students with above-average technical literacy.
- **Small sample size:** limits statistical confidence, especially for SUS scoring.
- **Demonstration context:** users may perform differently in real trip planning over longer time horizons.

Despite these limitations, the objective task results and consistent qualitative themes together support the conclusion that the core objectives were met.

Chapter 9

Conclusions and Future Work

This chapter summarises achievements, critiques the system against its goals, discusses limitations, and proposes future work.

9.1 Conclusions

GiTrip demonstrates that Git-like version control semantics can be meaningfully applied to collaborative trip planning. The completed system meets the Must-have objectives from the Project Specification:

- repository-based trip storage with commits and branches,
- constraint-aware automatic scheduling integrating routing data,
- interactive timeline and map editing with commit saving,
- three-way merge with a dedicated conflict-resolution UI.

Evaluation results provide objective evidence that users can successfully generate feasible itineraries and resolve merge conflicts within target times. Survey feedback confirms perceived novelty and practical value, particularly for group trips and scenarios requiring careful coordination.

9.2 Critical Reflection and Lessons Learned

Several lessons emerged during development and evaluation:

- **Git concepts are powerful but cognitively expensive.** During Evaluation Day, even participants who understood the UI hesitated when encountering terms like branch and merge. The Easy mode toggle proved essential, but discoverability and mental model support remain ongoing work.
- **Structured merge is domain-specific.** Early iterations used `JSON.stringify`-based comparison, which caused subtle bugs when key ordering differed between branches. Switching to deep structural equality, stable IDs, and explicit deletion semantics resolved these issues but required substantial reworking of the merge engine.

- **Constraint scheduling requires explainability.** Feedback from testing showed that users trusted the auto-scheduler more when adjustments (e.g., opening-hours nudges) were visibly annotated with reasons.
- **External dependency management matters.** During development, OpenRouteService rate limits and occasional Google Directions failures highlighted the need for Haversine-based fallback strategies to keep the planner usable under degraded conditions.

9.3 Limitations

Key limitations include:

- **Accessibility of interactive components:** maps and drag-based timelines remain challenging for keyboard-only and screen-reader users.
- **Merge granularity trade-offs:** while field-level conflicts are supported for selected fields, a fully general per-field resolver could overwhelm users.
- **No real-time collaboration:** GiTrip supports asynchronous collaboration via branches and merges, but not simultaneous live editing (CRDT/OT).
- **Heuristic scheduling:** the planner prioritises feasibility and responsiveness over global optimality.

9.4 Future Work

Future work opportunities are substantial:

- **Improved onboarding and guidance:** in-product tutorials for branching/merging, contextual tips, and clearer "Save" semantics to reduce confusion.
- **Accessibility enhancements:** keyboard-operable alternatives for timeline edits, text-based route editing, and stronger ARIA support.
- **Real-time collaboration:** explore CRDT-based shared plan editing, while preserving a mergeable history model.
- **Richer merge resolution:** expand conflict detection to additional fields, include conflict previews, and support partial merges.
- **Routing extensions:** support ferries/planes as explicit legs, integrate GTFS-based transit where applicable, and improve offline routing approximations.
- **Optional ML/AI extensions:** while excluded from the MVP, future versions could use machine learning to:
  - learn user preferences (pace, activity types) and suggest better defaults,
  - predict realistic dwell times and travel-time uncertainty,

- recommend destinations based on context (with careful privacy design).

These ideas should be implemented cautiously to maintain explainability and avoid undermining deterministic feasibility guarantees.

Author's Assessment of the Project

What is the (technical) contribution of this project? GiTrip contributes a practical system that applies Git-style branching, committing, and three-way merging to structured itinerary data, integrated with constraint-aware scheduling and interactive timeline/map editing.

Why is this contribution relevant to Computer Science? The project connects software configuration management concepts (history, merging, conflict resolution) with human-centred planning, demonstrating how domain semantics change the design of storage, algorithms, and user interfaces.

How can others make use of the work? Developers can reuse the snapshot-based repository model, merge engine patterns for structured JSON, and the planner's constraint-handling approach in other collaborative planning domains.

Why should this project be considered an achievement? It delivers an end-to-end deployed application combining routing integration, a scheduler, and a merge UI; evaluation shows users can complete core tasks successfully with limited guidance.

What are the limitations? The main limitations are accessibility for interactive controls, dependence on external routing providers, and the absence of real-time collaboration and AI-based recommendation within the MVP scope.

Appendix A

Evaluation Day Survey Instrument and Results

This appendix contains supporting evidence from the Evaluation Day survey. The main body of the report presents the key findings; the complete survey export is included here to allow assessors to inspect the full distribution of responses.

A.1 Survey Questions (Summary)

The survey included:

- Git familiarity and feature usage questions,
- System Usability Scale (SUS) items,
- Git concept understanding Likert items,
- Core functionality Likert items,
- Overall impression Likert items,
- free-text questions on positives, frustrations, adoption likelihood, and comparison to current workflows.

A.2 Full Survey Export PDF

Compilation note: place `eval_day_survey_results.pdf` in the same directory as this `.tex` file, or update the path below.

Appendix B

User Acceptance Test Results (Full Tables)

This appendix repeats the UAT tables from Chapter 8 for archival completeness. These results constitute objective evidence of feasibility and merge usability under time constraints.

Appendix C

Additional Diagrams and Screenshots

TODO: screenshot of the merge conflict resolution UI showing ours/theirs choices and the resolve button.

Figure C.1: Merge conflict resolution UI.

TODO: screenshot of the timeline editor with draggable stop blocks showing arrival/departure times.

Figure C.2: Timeline editor with draggable stop blocks.

TODO: screenshot of the map editor showing numbered markers, route polylines, and transit leg styling.

Figure C.3: Map editor with numbered markers and route geometry.

TODO: screenshot comparing Easy mode (simplified controls) vs Git mode (branches, commits, merge visible).

Figure C.4: Easy mode vs Git mode comparison.

TODO: screenshot of the algorithm visualisation page showing the map, pseudocode highlighting, and playback controls.

Figure C.5: Algorithm visualisation page.

TODO: screenshot of the quick explore mode showing the optimised route with summary stats.

Figure C.6: Quick explore mode with optimised route.

Appendix D

Project Schedule

Figure D.1 shows the project timeline across the two terms of the 2025/26 academic year. Table D.1 provides a breakdown of deliverables per phase.

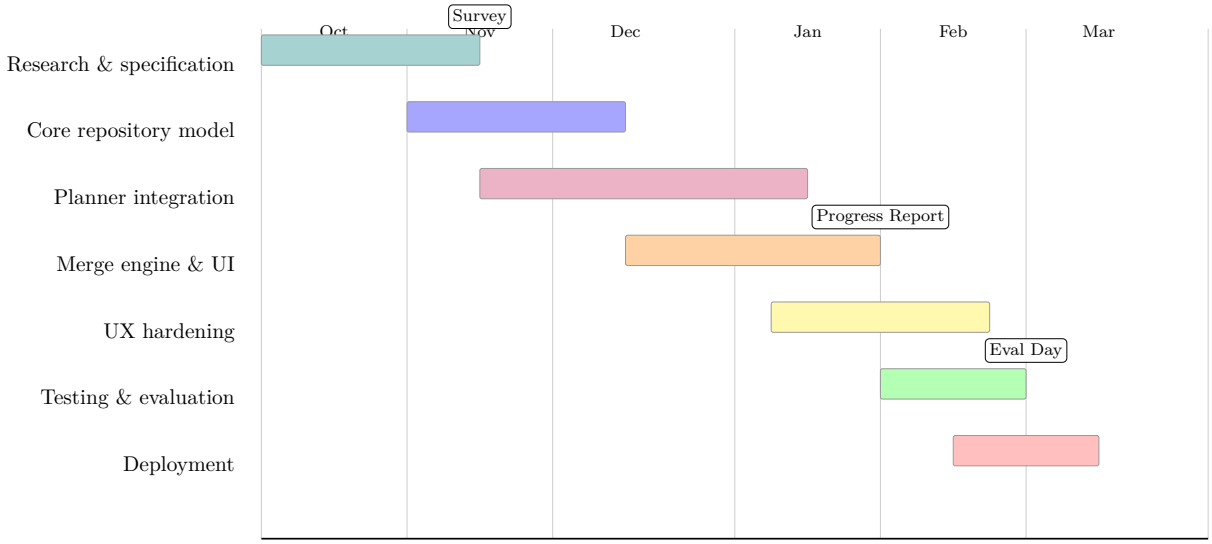


Figure D.1: Project Gantt chart showing the timeline for each development phase. Phases overlap where integration dependencies allowed parallel work. Key milestones are marked above the relevant bars.

Table D.1: Schedule detail: deliverables and status per phase.

Phase	**Deliverables**	**Status**
Research & specification	Competitor analysis; MoSCoW objectives; survey	Complete
Core repository model	Repos/branches/commits; snapshot storage	Complete
Planner integration	Routing matrix; ordering; constraints; spillover	Complete
Merge engine & UI	Three-way merge; conflicts; resolver UI	Complete
UX hardening	Easy mode; discoverability; visual polish	Complete
Testing & evaluation	Unit tests; UAT; survey analysis	Complete
Deployment	Containerisation; hosted instance; security checks	Complete